

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Assessment Tool Weightage: 20%

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Case Study

- Case 1: A food delivery company wants to deploy a cloud-native backend with web APIs, browser clients, and shared cloud storage for receipts and delivery images. Propose the architecture and explain the role of each component.
- Case 2: A media startup must decide between SAN-backed storage, object storage, and serverless processing for video upload workflows. Analyze the case and recommend a deployment model.
- Case 3: A smart city platform needs low-latency decision making near sensors while still using the cloud for analytics. Analyze the use of edge computing and fog computing in this environment.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is	Reasonable	Basic solution	Unclear solution

		feasible, practical, and well justified	solution with minor gaps		
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study — Questions & Answers

COURSE NAME: Cloud Computing and Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED: CO3: Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)

Assessment Tool Weightage: 20%

Bloom's Taxonomy: Precision (Level 3)

Question Count: 3

Total Marks: 30

Case 1

A food delivery company wants to deploy a cloud-native backend with web APIs, browser clients, and shared cloud storage for receipts and delivery images. Propose the architecture and explain the role of each component.

ANSWER

Proposed Architecture

- Client Layer: customer web app, delivery-partner mobile app, restaurant dashboard — all browser/HTTP-based clients.
- API Gateway: single entry point handling routing, authentication (JWT/OAuth), rate limiting, and TLS termination.
- Web API / Microservices Layer: separate REST services for Orders, Payments, Delivery Tracking, and Notifications — independently scalable.
- Cloud Storage: an object storage bucket holds receipts (PDF/JSON) and delivery-proof images, referenced by URL from the database rather than stored inline.
- Database: managed relational/NoSQL database for orders, users, and status — separate from the object storage used for binary assets.
- Serverless functions: triggered on new receipt/image uploads to generate thumbnails, validate file type, and update order status.
- Monitoring: centralized logging/metrics across gateway, services, and storage to track order latency and failures.

Role of Each Component

Component	Role
Browser / mobile client	User-facing interface; issues REST calls to place orders, track delivery, view receipts
API Gateway	Single secure entry point; authentication, routing, throttling
Web API services	Business logic for orders, payments, tracking — independently deployable/scalable
Object storage	Durable, scalable store for receipts and delivery images, decoupled from the database
Database	Structured, transactional data: order records, user accounts, status
Serverless functions	Event-driven processing (image resize, receipt generation) without

Component	Role
	dedicated servers
Monitoring	Observability into performance and errors across the whole chain

Justification

Separating structured order data (database) from binary receipts/images (object storage) keeps the database fast and small while storage scales independently and cheaply. Microservices behind an API gateway let the Orders and Delivery-Tracking services scale independently during peak meal-time demand, and serverless functions handle the bursty, event-driven work of processing each new receipt/image without idle server cost.

Case 2

A media startup must decide between SAN-backed storage, object storage, and serverless processing for video upload workflows. Analyze the case and recommend a deployment model.

ANSWER

Comparison

Option	Strengths	Weaknesses	Fit for video upload workflow
SAN-backed storage	Very low latency, high IOPS, good for active editing/transcoding workloads	Expensive, harder to scale elastically, capacity-bound	Good for active transcoding/edit stage, not bulk archive
Object storage	Massive scalability, low cost per GB, durable, native event triggers	Higher per-request latency than block storage	Ideal for storing raw uploads and final published videos
Serverless processing	Auto-scales with upload spikes, pay-per-use, integrates with storage events	Cold-start latency, execution time limits for long transcodes	Ideal for triggering transcoding/thumbnail jobs on upload

Recommended Deployment Model

- Ingest: users upload raw video directly to an object storage bucket (scales to any volume, cheap, durable).
- Trigger: the upload event fires a serverless function that queues a transcoding job (handles unpredictable upload bursts without idle infrastructure).
- Processing: for heavy, low-latency transcoding, a small SAN-backed compute cluster (or GPU instances with attached block storage) performs the actual encoding, since transcoding is I/O- and compute-intensive.
- Delivery: finished renditions are written back to object storage and served via a CDN.

Justification

This is a hybrid model: object storage for elastic, cheap ingest/archive and final delivery; serverless for event-driven orchestration; SAN-backed compute only where raw transcoding performance genuinely needs low-latency block I/O. Using SAN for everything would be cost-prohibitive at scale, while using only object storage and serverless would struggle with sustained, compute-heavy transcode jobs.

Case 3

A smart city platform needs low-latency decision making near sensors while still using the cloud for analytics. Analyze the use of edge computing and fog computing in this environment.

ANSWER

Analysis

- Requirement: low-latency decisions near sensors (traffic signals, public-safety cameras, environmental sensors) while still feeding centralized cloud analytics.
- Edge layer: lightweight inference running directly on/near sensors (e.g., a camera detecting an incident) so an immediate local action (alert, signal change) happens in milliseconds without a round trip to the cloud.
- Fog layer: neighborhood/district gateways aggregate multiple edge streams, correlate events across sensors (e.g., coordinating several traffic signals or detecting a city-block-wide pattern), and filter/compress data before forwarding it.
- Cloud layer: receives the aggregated, pre-processed data from fog nodes for city-wide dashboards, historical trend analysis, and machine-learning model retraining — tasks that are not latency-sensitive.

Why This Layering Works

Layer	Latency Need	Task Handled
Edge	Milliseconds	Immediate local detection/reaction at the sensor
Fog	Seconds	Cross-sensor correlation and regional coordination
Cloud	Minutes–hours (batch)	City-wide analytics, long-term storage, model training

Conclusion

Edge computing satisfies the hard real-time requirement by acting where data is generated; fog computing bridges the gap by coordinating groups of edge devices regionally; the cloud remains valuable for analytics that can tolerate latency. Relying on the cloud alone would violate the low-latency requirement, while relying on edge alone would prevent city-wide insight — the three-tier design is necessary, not optional.